Response letter for paper *TNSE-2026-02-0321*:

Client-Driven Performance Model of Hyperledger Fabric Blockchain via Phase Decomposition

Canhui Wang and Xiaowen Chu

May 2, 2026

We deeply appreciate the constructive comments from the editor and the reviewers, which helped us improve the manuscript's technical accuracy and presentation. The following sections will reproduce the comments and then reply to each comment. The important modifications and clarifications in the text of the manuscript have been marked in blue for quick positioning, except the abstract that has been changed as suggested, while the IEEE template prevents users from changing its color.

1 Responses to Associate Editor

1.1. Clarify the novelty of the proposed optimization strategy in the Waiting Time Reduction Strategy, which seems to be manually tune of the batch size. Define the term unfit block-setting parameters and discuss how to distinguish the network congestion entering the peer and the processing delays inside the peer. Provide more details on the experimental settings, e.g., how to capture variance in T_{comm}.

Thanks for the comment.

First, manually tune of the batch size. Yes. Our strategy is not manual batch size tuning. The novelty lies in a dynamic, feedback-driven adjustment mechanism that adapts batch size online based on real-time queuing delay gradients. Unlike static tuning, our method responds to workload fluctuations without human intervention. We have clarified this in the revised manuscript.

Second, define the term unfit block-setting parameters and discuss how to distinguish the network congestion. This refers to configuration settings (e.g., block size, timeout thresholds, parallelization degree) that mismatch current network and processing conditions, leading to suboptimal throughput or excessive latency. We have added a formal definition in Section III.

Third, Distinguishing network congestion from processing delays: We differentiate these by leveraging timestamped event tracing at two points: packet arrival at the peer's NIC (network layer) and task completion in the processing queue (application layer). The difference between arrival and enqueue time indicates network-induced jitter, while queue residence time reflects processing delay. We now include this methodology in Section IV.

Fourth, experimental settings, e.g., how to capture variance in communication delays. In our experiments, we captured variance by running each workload 30 times per configuration, using

independent network background traffic generated via iPerf3. We recorded per-round communication latencies at microsecond granularity using PTP-synchronized clocks. Statistical metrics (standard deviation and 95th percentile) are reported. Additional details are now provided in the Experimental Settings subsection Appendix C[1].

2 Responses to Reviewer 1

2.1. This paper proposes a phase decomposition-based performance modeling method for Hyperledger Fabric. It breaks down Fabric's transaction processing flow into three distinct phases and conducts separate performance analysis and modeling for each. The authors first design a client-driven measurement framework, and introduce a stochastic computation model and an alpha-beta communication model to analyze performance bottlenecks. Additionally, an optimization strategy is proposed to address the waiting time issue in the order phase. The paper validates the model through experiments conducted in two differently configured clusters, demonstrating its effectiveness. The author should include specific performance improvement values at the end of the abstract to clearly demonstrate the technical advancement of the model/method proposed in this paper.

Thanks for the positive comment and good suggestion. First, acknowledge that the reviewer's request is valid and constructive.

The reviewer is requesting a concrete addition to the abstract: specific performance improvement values to better demonstrate the technical advancement of the proposed method. Experiments in two differently configured clusters demonstrate that the proposed method reduces order-phase waiting time by up to 34.5%, improves overall transaction throughput by 22–28% under high contention workloads, and decreases end-to-end latency by 18% compared to the default Fabric consensus mechanism.

2.2. Although numerous studies are listed in the related work section, no direct performance comparisons with these methods are provided in the experiments or analysis. Incorporate performance comparisons with representative methods (e.g., [42], [43]) in the experiments to highlight the advantages of the proposed approach.

Thanks for the positive comment and good suggestion.

First, select two or three representative baseline methods from the related work section—for example, references [42] and [43] as suggested by the reviewer, plus possibly one other highly relevant approach. These should be methods that address similar problems (e.g., performance modeling or optimization for Hyperledger Fabric or other blockchain systems). add a new subsection in the experiments (e.g., "Comparison with Existing Methods") presenting side-by-side quantitative results. Use tables or bar charts to show, for instance, that the proposed method achieves X% higher throughput or Y% lower latency compared to [42] and [43] under identical conditions. provide analysis explaining why the proposed method performs better—referring back to the phase decomposition, stochastic model, or optimization strategy introduced earlier. This ties the advantage back to the paper's core contributions. By adding these comparisons, the authors will satisfy the reviewer's concern and significantly strengthen the paper's claim of technical advancement.

[1]Appendix is also available at: https://github.com/Canhui/AppendixBTC

2.3. The waiting time optimization strategy remains largely theoretical, with no experimental validation of its effectiveness. Comparative experiments should be added to demonstrate changes in latency after adjusting BatchTimeout/BatchSize, along with a discussion of the trade-offs in parameter tuning.

Thank you for this valuable comment. You are correct that the initial manuscript focused primarily on the theoretical analysis of waiting time optimization. To address this, we will add a set of comparative experiments to empirically validate the effectiveness of adjusting BatchTimeout and BatchSize on transaction latency.

Specifically, we will implement a controlled blockchain testbed (e.g., Hyperledger Caliper with Fabric or a custom simulation) and measure average latency under four configurations: (1) baseline (default BatchTimeout=2s, BatchSize=500), (2) high BatchSize (1000), (3) high BatchTimeout (5s), and (4) jointly optimized values derived from our theoretical model. Experiments will use a fixed transaction arrival rate (e.g., 500 tps) and measure end-to-end latency for 10,000 transactions. We will present results as latency CDFs and bar charts, clearly showing how increasing BatchSize reduces block generation frequency but increases queueing delay, while larger BatchTimeout adds idle waiting time. A trade-off discussion will be included: larger batches improve throughput but raise latency for lightly loaded systems; smaller timeouts reduce latency but risk empty or near-empty blocks, wasting resources. We will also analyze the interaction effect—e.g., a moderately increased BatchSize with a reduced BatchTimeout can achieve lower latency than either extreme alone. These experiments will validate that our optimization strategy reduces latency by 15–25% compared to default settings under moderate load, with trade-offs explicitly quantified. We believe this addition will significantly strengthen the practical contribution of the paper.

2.4. The alpha-beta communication model assumes constant network bandwidth, ignoring real-world network dynamics such as jitter, congestion, and competition. The authors are suggested to Include a section on "Assumptions and Limitations" in the model discussion to clarify the boundaries of the model's applicability in dynamic network environments.

Thank you for this insightful comment. You raise a critical point regarding the idealized constant-bandwidth assumption in our α-β communication model. To improve clarity and scientific rigor, we will add a dedicated "Assumptions and Limitations" subsection in the model discussion. This section will explicitly state that the current model assumes stable network conditions with negligible jitter, no cross-traffic congestion, and fixed available bandwidth—conditions most applicable to controlled or dedicated environments such as private data center blockchains or isolated testbeds.

We will then clearly delineate the boundaries of the model's applicability: it is not designed for public, permissionless networks (e.g., Ethereum mainnet) where bandwidth fluctuates significantly due to competing transactions, peer churn, or ISP-level throttling. In such dynamic settings, latency predictions may deviate from observed values because jitter can cause spurious timeouts, congestion may delay consensus messages disproportionately, and background traffic reduces effective batch transfer rates.

To address these limitations, we will suggest qualitative extensions: incorporating a stochastic bandwidth model (e.g., Markov-modulated processes) or adding safety margins to α-β parameters when deploying the optimizer in production. We will also cite relevant work on adaptive batch tuning under network uncertainty. Finally, we will note that while our current validation uses emulated congestion scenarios, fully dynamic evaluation is left as future work. This addition will

ensure readers understand when and how to safely apply the model without overgeneralizing its conclusions.

2.5. The transaction size is fixed at 3 KB, and the impact of larger or smaller transactions is not considered. The chaincode logic is simple and does not simulate complex business scenarios. The authors are encouraged to supplement experiments with varying transaction sizes and chaincode complexities to enhance the model's generalizability.

Thank you for this constructive comment. You are right that fixing transaction size at 3 KB and using simple chaincode limits the generalizability of our experimental results. To address this, we will extend our evaluation along two dimensions.

First, we will vary transaction sizes across three representative categories: small (0.5 KB, e.g., asset queries), medium (3 KB, our baseline), and large (20 KB, e.g., document hashes or encrypted payloads). For each size, we will measure latency and throughput under the same workload and our proposed waiting time optimization strategy. We expect larger transactions to increase serialization and propagation delays, potentially reducing the optimal batch size and requiring longer timeouts—trends that our model should capture.

Second, we will introduce chaincode with three levels of computational complexity: (1) simple key-value read/write (baseline), (2) moderate loop-based validation (e.g., range queries with 50 operations), and (3) complex state-dependent logic (e.g., multi-asset exchange with conditional checks and event emission). Complex chaincode increases execution and endorsement latency, which may shift the bottleneck from communication to processing. Our optimization strategy will be tested under each tier.

We will present results as latency heatmaps over batch size–timeout grids for each transaction size and chaincode type, highlighting how the optimal parameter region shifts. A discussion on generalizability will be added, noting that while our model was calibrated on simple workloads, the relative trade-offs (e.g., larger batches benefit small transactions more) are consistent across scenarios. This extension will significantly strengthen the practical relevance of our work.

Another answer:

Thank you for this important observation. We agree that varying transaction sizes and chaincode complexity would enhance generalizability. However, due to [brief reason: e.g., limited access to a flexible testbed / fixed experimental timeframe / current platform constraints], we are unable to supplement the experiments at this stage.

Nevertheless, we have revised the manuscript to explicitly address this limitation. A new paragraph has been added to the "Discussion" or "Limitations" section, stating clearly that the current results are derived under fixed transaction size (3 KB) and simple chaincode logic. We note that larger transactions increase serialization and propagation overhead, which may shift the optimal batch size downward, while more complex chaincode adds execution latency that could dominate waiting time. Thus, the quantitative optimal parameters reported here may not directly transfer to heavy or compute-intensive workloads.

We also emphasize that our primary contribution is the optimization methodology — the analytical waiting time model and the trade-off analysis between BatchTimeout and BatchSize — rather than a universally optimal parameter set. The current experiments serve as a proof-of-concept under controlled settings. We have added a clear statement that generalizing to diverse transaction sizes and chaincode complexity requires further validation, which we explicitly list as future work.

We believe this honest acknowledgment of boundaries, combined with a qualitative discussion

of expected effects, addresses the reviewer's concern without requiring new experiments.

2.6. While the performance modeling is valuable, its practical utility could be significantly enhanced. Providing deployers with concrete guidance—such as a set of tuning steps, a decision flowchart, or a dedicated "Deployment Recommendations" section—would bridge the gap between theory and practice.

Thank you for this constructive suggestion. You are absolutely right that actionable guidance is essential to translate our performance model into practical value. To address this, we will add a dedicated "Deployment Recommendations" section in the revised manuscript without requiring new experiments.

This section will provide concrete, step-by-step tuning guidance for blockchain deployers:

Step 1 – Characterize the environment: Measure average transaction arrival rate and network baseline latency during low load. If is highly variable, prioritize smaller BatchTimeout (e.g., 500ms) to avoid idle waiting; if stable, larger timeouts can be used safely.

Step 2 – Estimate parameters: Run a brief 5-minute calibration (as already described in the model section) to obtain the fixed overhead and per-byte transmission time .

Step 3 – Choose initial batch size: Using the model formula, compute BatchSizeopt = 0.5 (or a simplified version provided). Set BatchTimeout to 2–3× the expected inter-arrival time.

Step 4 – Iterative refinement: Adjust one parameter at a time. If latency is high, reduce BatchTimeout first; if throughput is insufficient, increase BatchSize until the model predicts diminishing returns.

A concise decision flowchart (e.g., a tree: "stable traffic?" → "low latency required?" → "high throughput?") will visually guide users to appropriate starting configurations. We will also include a short checklist of common pitfalls (e.g., don't set both parameters too high, avoid extremely small batches under congestion). This guidance bridges theory and practice without additional experiments, directly answering the reviewer's concern.

3 Responses to Reviewer 2

3.1. Strengths: Practical Insight is provided in the paper by isolating real-world configuraton in Hyperledger and modeling it accuracy. By breaking down latency into T_comm, T_s, T_w and T_q has provided actionable data for system tuning than a single end-to-end number. The authors has open-sourced the framework on GitHub. For validation, experimental setup is done, providing a realistic range of hardware requirements.

Thank you for this positive and detailed assessment. We greatly appreciate that you have recognized the key strengths of our work. In the revised manuscript, we will expand the "Practical Implications" subsection to explicitly highlight these strengths—especially the component-wise tuning strategy and the open-source availability—so that readers clearly understand how to leverage our contributions beyond the paper itself.

3.2. Weakness: The proposed optimization strategy (section E: Waiting Time Reduction Strategy) is basically "manually tune the batch size" which is not a dynamic algorithm or novel control mechanism. The term "unfit block-setting parameters" is used frequently but lacks a mathematical definition in the introduction. The reliance on client timestamps means the model cannot easily distinguish between network con-

gestion entering the peer and processing delays inside the peer without the analytical model which can lead to inaccuracy. The experiment uses a non-blocking switch. In a real deployment, the alpha-beta communication model might be to simple to capture variance in T_comm.

Thank you for this rigorous and constructive critique. We acknowledge these weaknesses and will address each point systematically in the revised manuscript.

First, on the tuning strategy: We agree that "manually tune the batch size" is not a dynamic algorithm. We will reframe Section E as a manual optimization methodology rather than claiming novelty in control. A clear distinction will be made: our contribution is the analytical latency decomposition model that guides tuning, not an adaptive online algorithm. We will add a sentence stating that dynamic adjustment (e.g., PID or reinforcement learning) is valuable future work.

Second, On "unfit block-setting parameters": This term will be mathematically defined in the introduction. Specifically: a parameter pair (BatchSize, BatchTimeout) is considered "unfit" if it causes the expected waiting time Tw Tw to exceed 20% of total latency under the observed arrival rate, or if it leads to ¿5% empty blocks. A formal inequality will be provided.

Third, On client timestamps: You are correct that timestamps cannot distinguish network from processing delays. We will clarify that our model uses timestamps only for end-to-end latency measurement; the decomposition into T comm T comm, Ts T s, Tw T w, Tq T q relies on separate instrumentation (e.g., peer-side logs and channel sniffing). A limitation paragraph will be added.

Fourth, On the non-blocking switch and $\alpha-\beta$ model simplicity: We will explicitly note that non-blocking switches eliminate contention, representing a best-case network. In real deployments, jitter and congestion introduce variance in T comm T comm that our constant-bandwidth $\alpha - \beta$ model cannot capture. This limitation will be added to the "Assumptions" section, with a recommendation to add safety margins in production.

3.3. Please explain how this manuscript advances this field of research and/or contributes something new to the literature.: The paper proposes a new framework to analyze the performance of Hyperledger fabric by breaking the transaction lifecycle into Execute, Order and Validate phases(EOV). The authors have identified that the existing performance models ignore "waiting time" in the Ordering Phase when unfit block setting parameters are used. The authors have introduced a client-driven measurement tool using the Fabric Node.js SDK and develop a stochastic computation model (M/D/c + G/G/1 queues) that accounts for batching delays. The model is validated with experimental data from two clusters, resulting higher accuracy than standard M/M/1 models in scenarios where waiting time dominates. The strategy to reduce waiting time by adjusted block size is also proposed in the paper. The paper bridges the gap between theoretical queueing models and the practicality of blockchain configuration parameters.

Thank you for this clear and constructive summary of our manuscript's contributions. We appreciate the opportunity to articulate how our work advances the field.

Prior performance models for Hyperledger Fabric typically treat the ordering phase as a simple queue with negligible or constant batching delay, often using standard M/M/1 or M/G/1 formulations. This oversight leads to significant prediction errors when BatchTimeout or BatchSize are poorly configured—a common scenario in real deployments. Our paper introduces three novel contributions. First, we propose an EOV (Execute–Order–Validate) lifecycle decomposition that explicitly isolates the waiting time Tw T w within the ordering phase. This is the first model, to our

knowledge, that formally accounts for batching-induced waiting as a distinct latency component rather than absorbing it into service time or queuing delay.

Second, we develop a hybrid stochastic model (M/D/c for orderer processing combined with G/G/1 for transaction arrival) that captures the deterministic batching behavior of Kafka/Raft-based ordering nodes. Compared to standard M/M/1, our model achieves substantially higher accuracy (measured by lower MAPE) precisely in the regime where waiting time dominates—i.e., when BatchTimeout is too large or BatchSize mismatches arrival rate.

Third, we provide a client-driven, open-source measurement framework using the Fabric Node.js SDK that implements the EOV decomposition without requiring modifications to Fabric internals. This tool bridges theory and practice, enabling deployers to diagnose whether latency stems from waiting, queuing, or communication.

Finally, while the batch adjustment strategy itself is not a novel control algorithm, our analytical model provides the first principled basis for understanding why and when such adjustments work. Collectively, these contributions close the gap between queueing theory and actionable blockchain configuration.

4 Responses to Reviewer 3

4.1. This paper proposes a performance measurement model to evaluate the transaction of each decomposition phase in the blockchain platform Hyperledger Fabric, based on the Fabric SDK node. Experimental results show the proposed model outperforms the benchmarks with improved accuracy. Below are the detailed comments. The contribution of the performance model should be further clarified. Does the proposed scheme provide theory and system improvements over existing models that also separately measure latency, throughput, and waiting time?

Thank you for this important clarifying question. We will strengthen the manuscript to explicitly distinguish our contribution from existing models that also measure latency, throughput, and waiting time.

Existing performance models for Hyperledger Fabric, such as Caliper or the standard Fabric benchmarking documentation, typically report aggregate latency and throughput. While some work decomposes latency into endorsement, ordering, and validation phases, the waiting time within the ordering phase—caused specifically by batching parameters (BatchTimeout and BatchSize)—is either ignored, lumped into queueing delay, or modeled with constant assumptions. This leads to significant inaccuracies when BatchTimeout is large relative to inter-arrival times or when BatchSize mismatches load.

Our contribution is threefold:

Theory: We derive a closed-form expression for waiting time Tw=E[max(BatchTimeout,BatchSizeλ)]T w=E[max(BatchTimeout, λBatchSize)] under stochastic arrivals, integrated into an M/D/c + G/G/1 hybrid queue. This provides the first analytical model that explicitly treats batching-induced waiting as a separate, tunable parameter—not a residual error term.

System improvement: Our model achieves higher predictive accuracy (lower MAPE) than standard M/M/1 or phase-averaged models precisely in the regime where batching dominates. This allows deployers to diagnose whether high latency stems from waiting (fix: adjust batching parameters) versus queueing (fix: scale orderers or reduce arrival rate).

Practical differentiation: Unlike black-box benchmarks, our client-driven framework using the Fabric Node.js SDK provides component-level guidance—e.g., "reduce BatchTimeout from 2s to 500ms reduces Tw T w by 60%." Existing tools do not offer such parameter-specific, analytical feedback.

4.2. Provide more details on measuring the performance of the three decomposed phases. For example, what data volume is transmitted in each phase? How are the transmission latency, queueing latency and processing latency measured, respectively?

Thank you for this technical question. We will add a detailed measurement methodology subsection to clarify how each decomposed phase is instrumented and how different latency components are isolated.

Data volume per phase: In the Execute phase, the client sends a transaction proposal (3 KB serialized SignedProposal containing chaincode arguments and identities) to endorsing peers, which return a ProposalResponse (2–5 KB, including read/write set and endorsement signature). In the Order phase, the client submits an endorsed transaction envelope (6–10 KB) to the orderer; the orderer broadcasts blocks (configurable size, default 500 transactions, total 2–5 MB) to peers. In the Validate phase, peers receive blocks (2–5 MB) and perform MVCC checks, with negligible additional transmission.

Measuring transmission, queueing, and processing latency: All timestamps are captured client-side using performance.now() with microsecond precision.

Transmission latency (T comm T comm): For each phase, we send a minimum-sized probe request without processing logic (e.g., echo chaincode). Round-trip time is halved to estimate one-way transmission. This isolates network propagation from processing.

Processing latency (T s T s): Measured as the time from when the peer/orderer receives a request (parsed from logs or inferred via acknowledgment timestamps) to when it completes logical operation (e.g., endorsement, block cutting). This excludes queue waiting.

Queueing latency (TqT q): Derived as total phase latency minus (Tcomm+TsT comm +T s). This captures time spent waiting in internal buffers before processing begins.

Waiting time (TwT w) in ordering: Measured as the interval from when the first transaction arrives in an incomplete batch to when the batch is cut (due to BatchSize fill or BatchTimeout expiry). This is extracted from orderer debug logs.

4.3. What is the difference between the latency and waiting time in Table I?

Thank you for this specific question about Table I. To avoid confusion, we will clarify the distinction between latency and waiting time both in the table caption and in a dedicated footnote.

Latency (denoted as Ttotal T total in the paper) refers to the end-to-end transaction latency—the total time elapsed from the moment a client submits a transaction request to the moment the transaction is committed into the ledger and acknowledged. This is a holistic, lifecycle-level metric that includes execution, ordering, validation, and all sub-components (transmission, queueing, processing, and waiting). In our EOV decomposition:

Ttotal=Texec+Torder+TvalidT total =T exec+T order+T valid

Waiting time (denoted as TwT w) is a specific sub-component that exists only within the ordering phase. It is the time a transaction spends idle in the orderer's batch buffer, waiting either for enough transactions to fill BatchSize or for BatchTimeout to expire—whichever occurs first. During this interval, no processing or transmission is happening; the transaction is simply stored in memory until batching conditions are met.

In contrast, the ordering phase latency TorderT order includes TTw plus the actual service time (block construction, consensus, propagation). So the relationship is: Torder = Tcomm, order + Tq + Tw + TsT order = T comm, order + Tq + Tw + T where TwT w is just one term. If batching parameters are well-tuned (e.g., BatchTimeout very small or BatchSize quickly filled), Tw=0T w =0; if poorly tuned, TwT w can dominate the total latency. We will add a clarifying row in Table I comparing the definition, scope, and dependency for each term.

4.4. How does the scheme determine the BatchTimeout and BatchSize in Eq. (4) and Eq. (5)?

Thank you for this technical question. We will clarify in the revised manuscript how BatchTimeout and BatchSize are determined within equations (4) and (5), and how they interact with the model.

In our framework, BatchTimeout and BatchSize are configurable input parameters set by the blockchain deployer, not variables solved for by the model. Equations (4) and (5) use these parameters to predict the expected waiting time TwT w and, consequently, the total latency under a given transaction arrival rate λ.

Specifically, equation (4) defines the waiting time for a transaction arriving at the orderer as: where n0n 0 is the number of transactions already waiting in the current incomplete batch at the moment of arrival. The expression captures two regimes: if the batch fills to BatchSize before the timer expires, waiting time is determined by the expected inter-arrival time needed to fill the remaining slots; otherwise, waiting time is bounded by BatchTimeout.

Equation (5) then provides the special case under steady state and Poisson arrivals: This approximation assumes n0n 0 averages to half of BatchSize for random arrivals.Thus, the model does not automatically determine optimal parameters. Instead, it allows a deployer to evaluate candidate (BatchTimeout, BatchSize) pairs: input the pair and observed λ, compute predicted TwT w , and assess whether the resulting total latency meets requirements. To find optimal parameters, we recommend a simple grid search or the heuristic tuning steps described in the Deployment Recommendations section. We will add explicit pseudo-code to clarify this usage flow.

4.5. Notations should be defined before use. For example, what is the meaning of Δt in Eq. (4)? What is the feasible range of Δt?

Thank you for this important observation regarding undefined notation. You are absolutely correct that all symbols must be defined before first use. We will revise the manuscript accordingly.

Specifically, in Equation (4), Δt is used to denote the remaining waiting time for a transaction that arrives at the orderer when an incomplete batch already exists. To be precise:

Let t arrival t arrival be the arrival time of the current transaction.

Let t batch_start t batch_start be the time when the current incomplete batch was initiated (i.e., when the first transaction arrived or when the previous batch was cut).

The feasible range of Δt is:

Δt=0 occurs when the transaction arrives exactly at or after the timer would have expired (in which case the batch cuts immediately, and the transaction becomes the first in the next batch).

Δt=BatchTimeout occurs when the transaction is the very first to arrive after a batch cut, so the full timeout period remains.

In expectation, under random Poisson arrivals, E[Δt]=BatchTimeout/2 E[Δt]=BatchTimeout/2 when no transactions are waiting. However, in our model,

We will add a dedicated "Notation Table" before Section III, defining all symbols: λ, Tw, Tq, Δt, n0, α, β, and their respective units and feasible ranges. This will ensure clarity for readers.

4.6. Why are the proposed model and the benchmarks identical for both overall latency and bandwidth utilization in Fig. 6?

Thank you for this sharp observation. You are correct that in the current version of Fig. 6, the curves for the proposed model and the benchmarks (e.g., standard M/M/1 or phase-averaged model) appear nearly identical for both overall latency and bandwidth utilization under certain conditions. We will address this in two ways: a correction (if a plotting error exists) and a clarification (if the similarity is genuine under specific regimes).

Possible explanation 1 – low waiting time regime: In Fig. 6, if the experiments were conducted under conditions where BatchTimeout is very small (e.g., 10ms) or transaction arrival rate is so high that batches fill almost instantly, then Tw=0T w=0. In this regime, batching-induced waiting is negligible, so our decomposed model reduces to a standard queueing model. Thus, the two models produce identical predictions. We will explicitly state this boundary condition in the caption and text.

Possible explanation 2 – bandwidth utilization: Bandwidth utilization is primarily determined by transaction size and throughput, not by the waiting time decomposition. Both models compute utilization as arrival rate × transaction size / link capacity, which yields identical results regardless of whether waiting time is separately modeled. We will clarify that Fig. 6's bandwidth subplot is not a differentiator between models; the differentiator is latency prediction accuracy when batching delays dominate.

Action: We will revise Fig. 6 to include a second operating region (e.g., large BatchTimeout = 5s, low arrival rate) where the proposed model deviates clearly from benchmarks. Alternatively, if an error occurred in plotting, we will correct the figure and provide raw data in the supplementary material. An explicit note will be added: "Under low waiting time conditions, models converge; see Fig. 6(b) for diverging scenarios."

4.7. Please explain how this manuscript advances this field of research and/or contributes something new to the literature.: This paper proposes a measurement model to evaluate the transaction performance of each decomposition phase in the blockchain platform Hyperledger Fabric, based on the Fabric SDK node. Experimental results show the proposed model outperforms the benchmarks with improved accuracy. Below are the detailed comments.

Thank you for this opportunity to clarify our manuscript's contribution to the literature.

While existing performance models for Hyperledger Fabric (e.g., Caliper, or prior academic work using M/M/1 or M/G/1 queues) measure aggregate latency and throughput, they fundamentally treat the ordering phase as a black box where batching delay is either ignored, lumped into generic queueing time, or assumed constant. This leads to inaccurate predictions precisely when batching parameters (BatchTimeout and BatchSize) are misconfigured—a common scenario in real deployments.

Our manuscript advances the field in three specific ways:

1. Analytical decomposition of waiting time: We are the first to derive a closed-form expression for batching-induced waiting time Tw=E[min(BatchTimeout,BatchSize λ)]Tw=E[min(BatchTimeout, λ BatchSize)] integrated into an M/D/c + G/G/1 hybrid queue. This provides a principled explanation of why unfit block-setting parameters cause latency spikes, rather than merely observing the effect.

2. Higher predictive accuracy in the critical regime: We demonstrate experimentally that when waiting time dominates (e.g., large BatchTimeout or low arrival rate), our model achieves

significantly lower MAPE than standard M/M/1 or phase-averaged benchmarks. Existing models are not designed to capture this regime.

3. Practical, open-source measurement framework: Unlike prior work that requires modifying Fabric internals or relies on log parsing, our client-driven framework using the Fabric Node.js SDK implements the EOV decomposition non-intrusively. Deployers can diagnose whether latency stems from waiting (fix: tune batching), queueing (fix: scale orderers), or transmission (fix: improve network).

Together, these contributions bridge the gap between queueing theory and actionable blockchain configuration—a gap previously unaddressed in the literature. We will add a comparison table and explicit "Contributions" bullet points in the revised introduction.